

Question description

Kaaleppi has just robbed a bank and is now heading to the harbor. However, the police wants to stop him by closing some streets of the city.

What is the minimum number of streets that should be closed so that there is no route between the bank and the harbor?

Constraints

- $2 \leq n \leq 500$
- $1 \leq m \leq 1000$
- $1 \leq a, b \leq n$

Input

The first input line has two integers n and m : the number of crossings and streets. The crossings are numbered $1, 2, \dots, n$. The bank is located at crossing 1, and the harbor is located at crossing n .

After this, there are m lines that describe the streets. Each line has two integers a and b : there is a street between crossings a and b . All streets are two-way streets, and there is at most one street between two crossings.

Output

First print an integer k : the minimum number of streets that should be closed. After this, print k lines describing the streets. You can print any valid solution.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

4 5
1 2
1 3
2 3
3 4
1 4

EXPECTED OUTPUT

2
1 4
3 4

Test Case 2

INPUT (STDIN)

4 5
1 1
2 2
2 3
3 4
1 4

EXPECTED OUTPUT

1
1 4

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void link(int i,int h)

Test Case 2

KEYWORD

int bfs(int n,int s,int t)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

735

Test Case 3

NLOC

93

Graph10.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAXN 505
6 #define MAXM 1005
7
8 int head[MAXN], to[MAXM * 4], cap[MAXM * 4], flow[MAXM * 4], nxt[MAXM * 4], edge_cnt;
9 int orig_u[MAXM], orig_v[MAXM];
10 int parent_edge[MAXN], parent_node[MAXN];
11 int visited[MAXN];
12
13 void link(int i, int h) {
14     to[edge_cnt] = h;
15     cap[edge_cnt] = 1;
16     flow[edge_cnt] = 0;
17     nxt[edge_cnt] = head[i];
18     head[i] = edge_cnt++;
19
20     to[edge_cnt] = i;
21     cap[edge_cnt] = 1;
22     flow[edge_cnt] = 0;
23     nxt[edge_cnt] = head[h];
24     head[h] = edge_cnt++;
25 }
```

```

26
27int bfs(int n, int s, int t) {
28    int q[MAXN], front = 0, rear = 0;
29    memset(visited, 0, sizeof(visited));
30    memset(parent_edge, -1, sizeof(parent_edge));
31
32    q[rear++] = s;
33    visited[s] = 1;
34
35    while (front < rear) {
36        int u = q[front++];
37        if (u == t) return 1;
38
39        for (int e = head[u]; e != -1; e = nxt[e]) {
40            int v = to[e];
41            if (!visited[v] && cap[e] - flow[e] > 0) {
42                visited[v] = 1;
43                parent_edge[v] = e;

```

```

44         parent_node[v] = u;
45         q[rear++] = v;
46     }
47 }
48 }
49 return 0;
50}
51
52int max_flow(int n, int s, int t) {
53     int total_flow = 0;
54     while (bfs(n, s, t)) {
55         for (int v = t; v != s; v = parent_node[v]) {
56             int e = parent_edge[v];
57             flow[e] += 1;
58             flow[e ^ 1] -= 1;
59         }
60         total_flow++;

```

```
61     }
62     return total_flow;
63 }
64
65 void mark_reachable(int s) {
66     int q[MAXN], front = 0, rear = 0;
67     memset(visited, 0, sizeof(visited));
68
69     q[rear++] = s;
70     visited[s] = 1;
71
72     while (front < rear) {
73         int u = q[front++];
74         for (int e = head[u]; e != -1; e = nxt[e]) {
75             int v = to[e];
76             if (!visited[v] && cap[e] - flow[e] > 0) {
77                 visited[v] = 1;
```

```

78         q[rear++] = v;
79     }
80 }
81 }
82}
83
84int main() {
85     int n, m;
86     if (scanf("%d %d", &n, &m) != 2) return 0;
87
88     memset(head, -1, sizeof(head));
89     edge_cnt = 0;
90
91     for (int i = 0; i < m; i++) {
92         scanf("%d %d", &orig_u[i], &orig_v[i]);
93         link(orig_u[i], orig_v[i]);
94     }

```

```
95
96     int k = max_flow(n, 1, n);
97     printf("%d\n", k);
98
99     mark_reachable(1);
100
101     for (int i = 0; i < m; i++) {
102         int u = orig_u[i];
103         int v = orig_v[i];
104         if ((visited[u] && !visited[v]) || (visited[v] && !visited[u])) {
105             printf("%d %d\n", u, v);
106         }
107     }
108
109     return 0;
110 }
111
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
4 5
```

```
1 2
```

```
1 3
```

```
2 3
```

```
3 4
```

```
1 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph10.exe
```

```
2
```

```
3 4
```

```
1 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
4 5
```

```
1 1
```

```
2 2
```

```
2 3
```

```
3 4
```

```
1 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph10.exe
```

```
1
```

```
1 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```